

Escritura en la API y documentación con OpenAPI

2. Escritura en la API y documentación con OpenAPI

Objetivo de la sesión

Pasar de “sé construir un CRUD” a “sé describir y probar su contrato”.

Ya sabes

- DTOs de entrada y salida
- Bean Validation y `@Valid`
- GET, POST, PUT y DELETE
- `ResponseEntity` y códigos HTTP



Ahora añades

- Contrato OpenAPI
- Documentación generada con springdoc
- Swagger UI como cliente HTTP
- `@Operation` y `@ApiResponse`s

El objetivo no es aprender otro CRUD: es entender cómo ese CRUD se convierte en un contrato utilizable por personas y herramientas.

El LibroController completo (1/2)

Primero, los dos endpoints de lectura que ya conoces:

```
@RestController
@RequestMapping("/api/v1/libros")
@RequiredArgsConstructor
public class LibroController {
    private final LibroService libroService;

    @GetMapping
    public ResponseEntity<List<LibroResponseDTO>> getAll() {
        return ResponseEntity.ok(libroService.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<LibroResponseDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(libroService.findById(id));
    }
}
```

El LibroController completo (2/2)

Los tres endpoints de escritura reutilizan las mismas ideas de HTTP, DTOs y validación:

```
@PostMapping
public ResponseEntity<LibroResponseDTO> create(@Valid @RequestBody LibroCreateDTO dto) {
    return ResponseEntity.status(HttpStatus.CREATED).body(libroService.create(dto));
}

@PutMapping("/{id}")
public ResponseEntity<LibroResponseDTO> update(@PathVariable Long id,
        @Valid @RequestBody LibroCreateDTO dto) {
    return ResponseEntity.ok(libroService.update(id, dto));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    libroService.delete(id);
    return ResponseEntity.noContent().build();
}
```

POST, PUT y DELETE: mismo patrón, distinto resultado

Cada operación utiliza el código de éxito que mejor describe lo que ha ocurrido.

POST

201 Created

Crea un recurso nuevo

JSON → LibroCreateDTO

PUT

200 OK

Reemplaza y devuelve el estado actualizado

id + JSON → LibroCreateDTO

DELETE

204 No Content

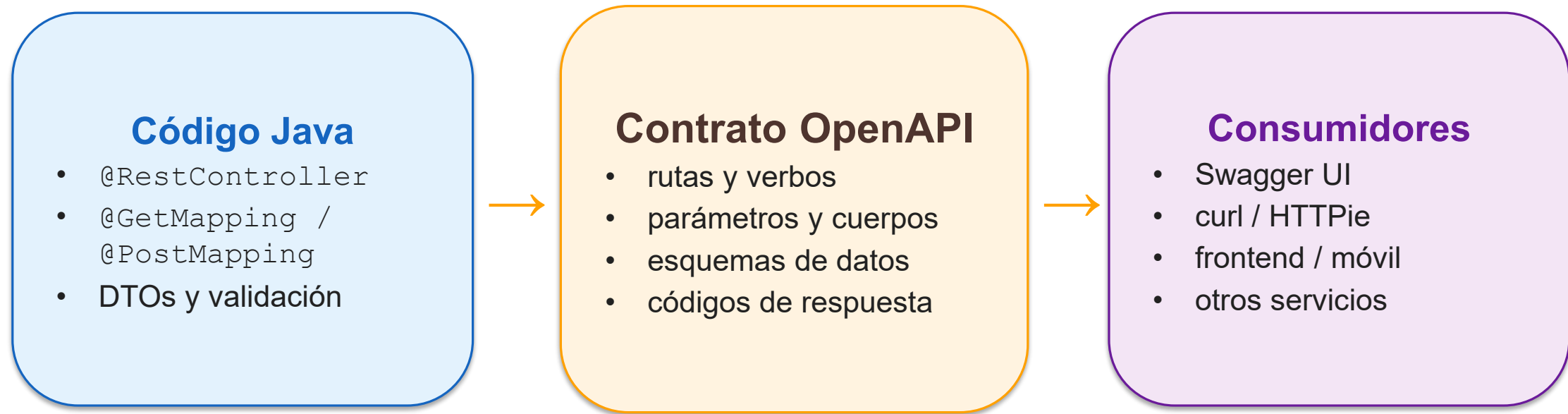
Elimina y no devuelve cuerpo

id en la ruta

@RequestBody convierte el JSON en un DTO · @Valid valida antes de ejecutar · ResponseEntity decide estado y cuerpo.

El contrato de una API

El consumidor necesita conocer la API sin leer tu código.

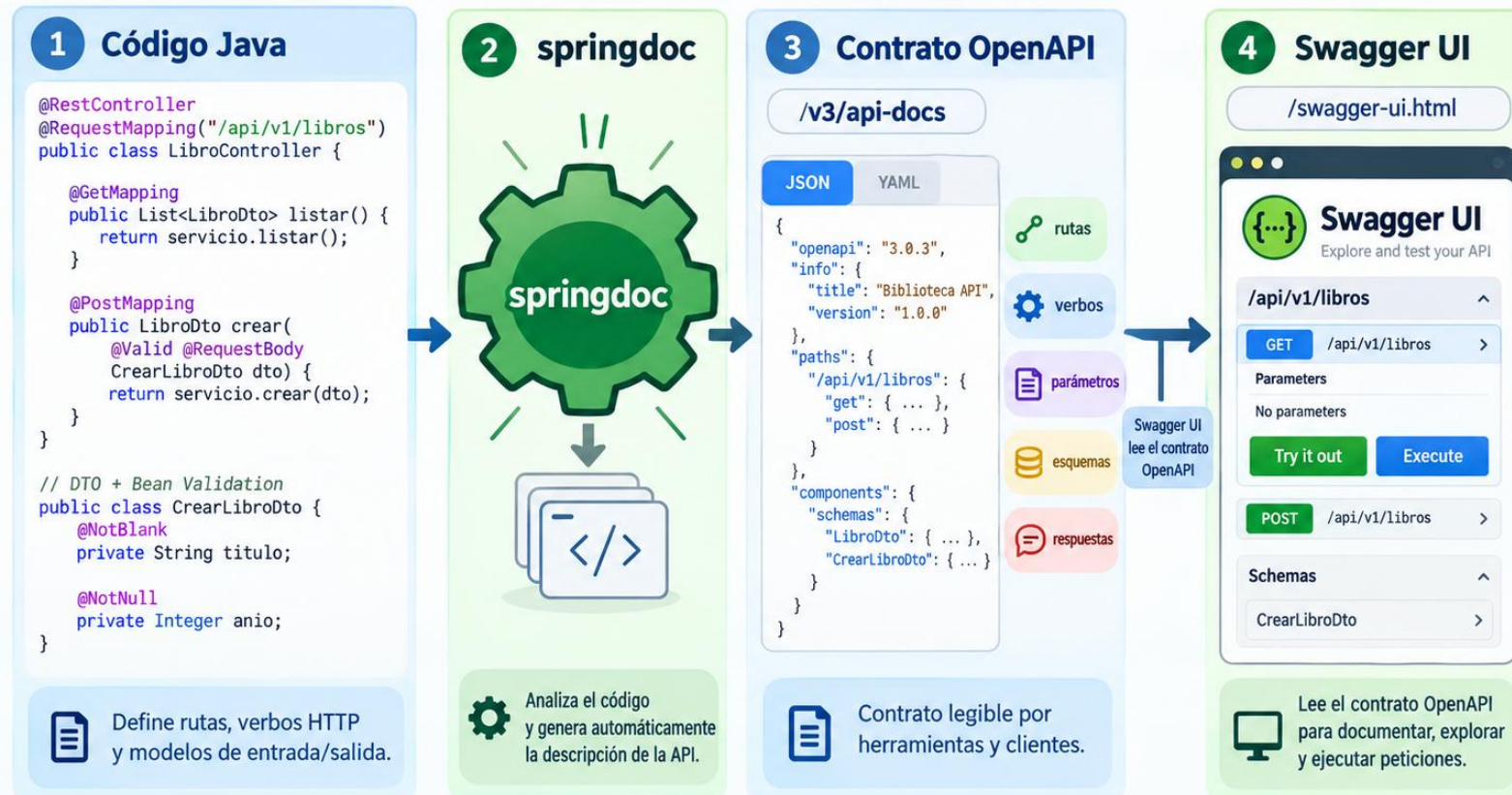


OpenAPI es una descripción formal y estándar de ese contrato; springdoc puede generar gran parte de ella desde el código.

Del código Java a OpenAPI y Swagger UI

Del controlador al contrato OpenAPI y Swagger UI

Cómo una API documenta sus endpoints y los expone para exploración y prueba



Idea clave: Swagger UI **no inspecciona directamente el controller;** lee el contrato OpenAPI generado por springdoc.

Swagger UI ejecuta peticiones HTTP reales

De Swagger UI a la petición HTTP real

Qué ocurre cuando pulsas Try it out y Execute

1 Elegir endpoint

Libros

POST

/api/v1/libros

Crea un nuevo libro.

Parámetros

No tiene parámetros.

Request body

Ejemplo de JSON:

```
{
  "titulo": "Clean Code",
  "autor": "Robert C. Martin",
  "anio": 2008
}
```

Seleccionas la operación y ves parámetros, esquema y ejemplo JSON.

2 Try it out

Request body

required

```
1 {
2   "titulo": "Clean Code",
3   "autor": "Robert C. Martin",
4   "anio": 2008
5 }
```

Try it out

Swagger UI habilita la edición de los datos de entrada.

3 Execute

Execute

POST

/api/v1/libros



Servidor API
(tu aplicación)

Se envía una petición HTTP real a tu aplicación.

4 Respuesta del servidor

Código de estado



201 Created

Cabeceras HTTP

Content-Type: application/json
Location: /api/v1/libros/42
Date: Wed, 21 May 2025 10:15:30 GMT
...

Cuerpo de la respuesta

```
{
  "id": 42,
  "titulo": "Clean Code",
  "autor": "Robert C. Martin",
  "anio": 2008
}
```

Swagger UI muestra el resultado real: estado, cabeceras y cuerpo.



Swagger UI

POST /api/v1/libros

HTTP real



Controller
(tu código)



Base de datos
(guarda la información)



Respuesta
(estado, cabeceras y cuerpo)



Idea clave

Swagger UI no inventa respuestas: actúa como cliente HTTP y ejecuta el mismo endpoint que usarías con curl o Postman.

Configurar springdoc y sus rutas

Una configuración mínima añade metadatos generales; springdoc genera el contrato y Swagger UI lo representa.

```
@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI libreriaOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Librería API")
                .version("v1")
                .description("API para gestionar el catálogo..."));
    }
}
```

/v3/api-docs

Contrato OpenAPI en JSON.

/swagger-ui.html

Punto de entrada a la interfaz
Swagger UI.

/documentacion

Ruta opcional configurada con
springdoc.swagger-ui.path.

Documentar respuestas con @Operation y @ApiResponse

```
@Operation(summary = "Crear un libro nuevo")
@ApiResponses({
    @ApiResponse(responseCode = "201",
        description = "Libro creado correctamente"),
    @ApiResponse(responseCode = "400",
        description = "Datos no válidos"),
    @ApiResponse(responseCode = "404",
        description = "Editorial no encontrada")
})
@PostMapping
public ResponseEntity<LibroResponseDTO> create(
    @Valid @RequestBody LibroCreateDTO dto) {
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(libroService.create(dto));
}
```

201

Libro creado

400

Datos no válidos

404

Recurso relacionado no encontrado

Estas anotaciones enriquecen el contrato; no ejecutan la lógica ni producen por sí solas esos códigos.

Código que ejecuta vs código que documenta

Código que ejecuta la API

Estas anotaciones y el código del método determinan el comportamiento real del endpoint.

```
@PostMapping
public ResponseEntity<LibroResponseDTO> create(
    @Valid @RequestBody LibroCreatedDTO dto) {
    LibroResponseDTO creado = libroService.create(dto);
    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(creado);
}
```



Resultado en tiempo de ejecución

- ✓ Qué lógica se ejecuta (`libroService.create(dto)`)
- ✓ Qué código de estado devuelve (201 Created)
- ✓ Qué cuerpo de respuesta se envía
- ✓ Qué validaciones se aplican (`@Valid`, Bean Validation)
- ✓ Qué excepciones pueden producirse (y cómo se gestionan)



Idea clave

Documentar un **código de respuesta** (por ejemplo, 404) no hace que el endpoint lo devuelva automáticamente. Las anotaciones de OpenAPI describen qué respuestas son posibles, pero el código del método es el que determina qué se devuelve realmente en cada caso.

Código que documenta la API

Estas anotaciones añaden **información al contrato OpenAPI**. No cambian el comportamiento del endpoint.

```
@Operation(
    summary = "Crea un nuevo libro",
    description = "Crea un libro en el sistema y devuelve el recurso creado."
)
@ApiResponses({
    @ApiResponse(responseCode = "201", description = "Libro creado",
        content = @Content(schema = @Schema(implementation = LibroResponseDTO.class))),
    @ApiResponse(responseCode = "400", description = "Datos no válidos",
        content = @Content()),
    @ApiResponse(responseCode = "500", description = "Error interno")
})
@PostMapping
public ResponseEntity<LibroResponseDTO> create(
    @Valid @RequestBody LibroCreatedDTO dto) {
    ...
}
```



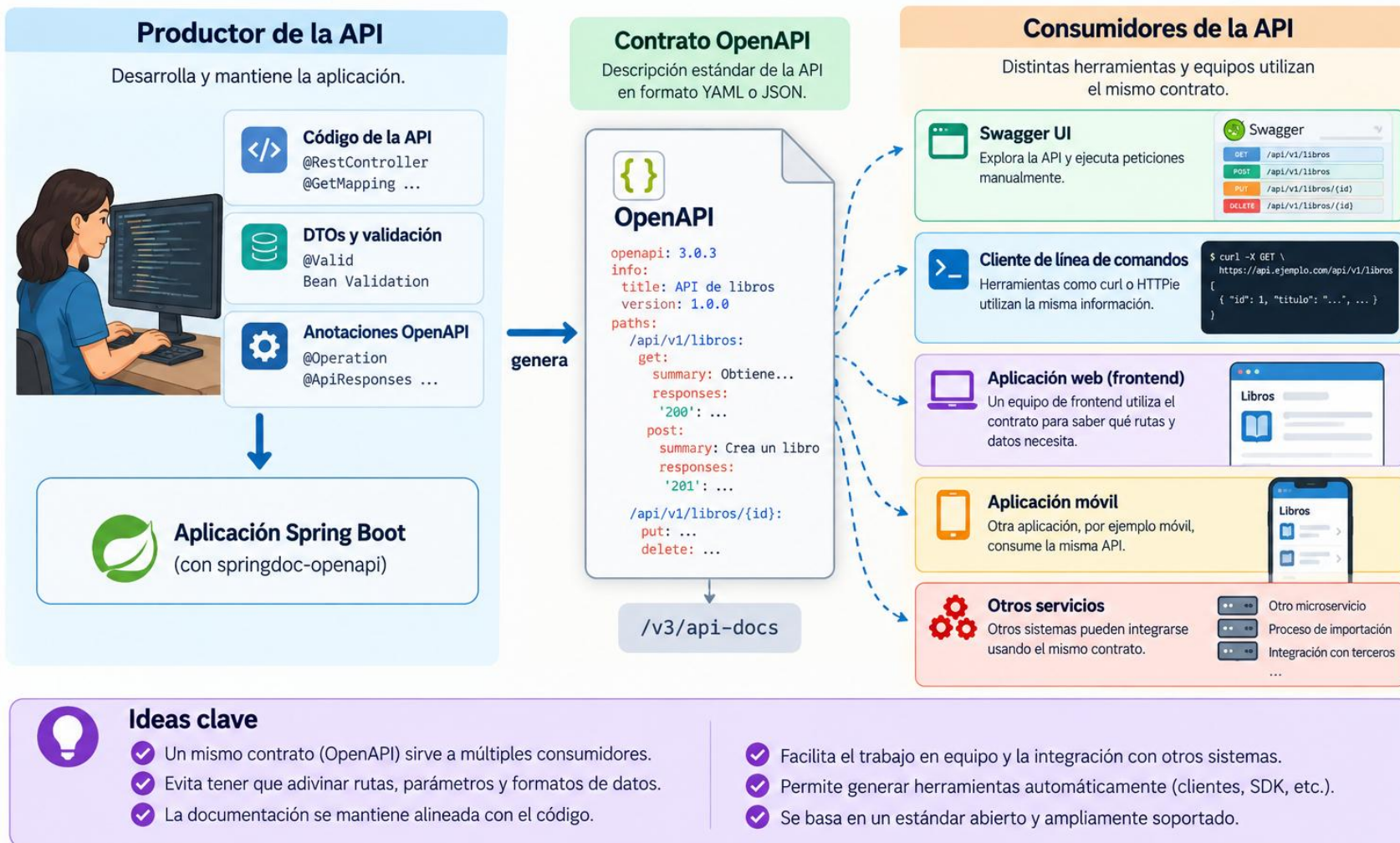
Resultado en la documentación (OpenAPI / Swagger UI)

- ✓ Descripción de la operación (summary y description)
- ✓ Códigos de respuesta documentados (201, 400, 500, ...)
- ✓ Esquemas de datos de entrada y salida (`LibroCreatedDTO`, `LibroResponseDTO`)
- ✓ Ejemplos y detalles adicionales (si se añaden)
- ✓ Se muestra en `/v3/api-docs` y en `/swagger-ui.html`

OpenAPI como contrato compartido

OpenAPI: un contrato común para muchos consumidores

Una misma descripción permite que distintos clientes conozcan y utilicen la API sin necesidad de leer su código.



Cierre de la sesión

Lo mínimo que debe quedar claro

- ✓ OpenAPI describe formalmente el contrato de una API.
- ✓ Springdoc genera automáticamente gran parte del contrato desde controllers y DTOs.
- ✓ Swagger UI lee /v3/api-docs y puede ejecutar peticiones HTTP reales.
- ✓ @Operation y @ApiResponse mejoran la documentación, no el comportamiento.
- ✓ El mismo contrato puede ser utilizado por personas, herramientas y otras aplicaciones.

Idea clave Próximo paso: utilizar el contrato y la documentación para probar y evolucionar la API con más seguridad.